

STANFORD CS149- ASSIGNMENT 1

Question 1:

Part 1)

Modify the starter code to parallelize the Mandelbrot generation using two processors. Specifically, compute the top half of the image in thread 0, and the bottom half of the image in thread 1. This type of problem decomposition is referred to as spatial decomposition since different spatial regions of the image are computed by different processors.

We implemented the parallelization by dividing the image into equal parts and assigning each part to a thread. The formula used is:

```
int rows = args->height / args->numThreads;
int startRow = args->threadId * rows;
int endRow;
if (args->threadId == args->numThreads-1){
    endRow=args->height; //all remaining rows
}
else{
    endRow=startRow + rows;
}
```

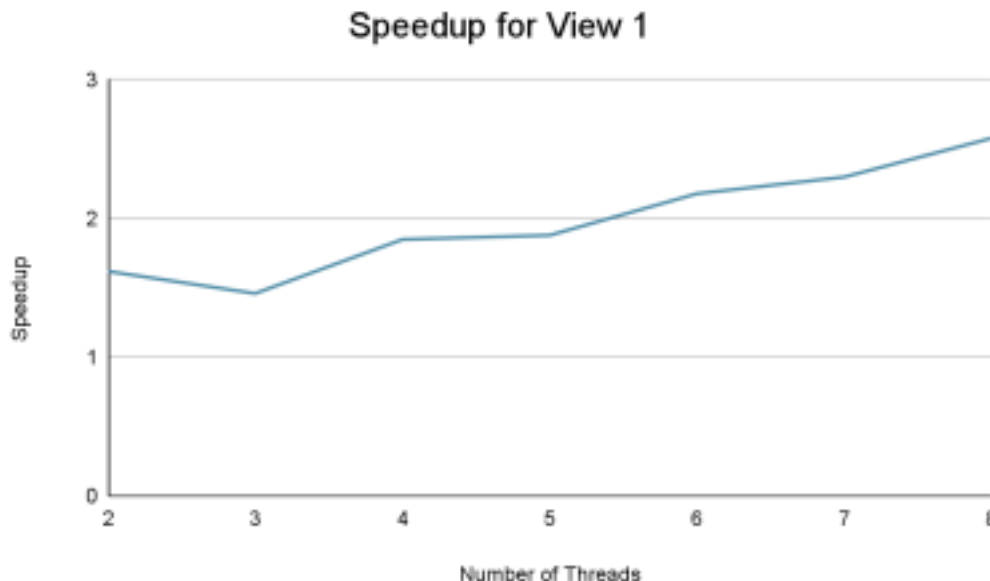
This calculates how many rows a single thread can handle by equally dividing the height by the number of threads. Then we calculate the starting row and the ending row for each thread, such that if the height is not perfectly divisible by the number of threads, all the remaining rows are handled by the last thread.

(This part was simple thread assignment so we did not take help from any external resource)

Part 2)

Extend your code to use 2, 3, 4, 5, 6, 7, and 8 threads, partitioning the image generation work accordingly (threads should get blocks of the image). Note that the processor only has four cores but each core supports two hyper-threads, so it can execute a total of eight threads interleaved on its execution contents. In your write-up, produce a graph of speedup compared to the reference sequential implementation as a function of the number of threads used FOR VIEW 1. Is speedup linear in the number of threads used? In your writeup, hypothesize why this is (or is not) the case? (you may also wish to produce a graph for VIEW 2 to help you come up with a good answer. Hint: take a careful look at the three-thread datapoint.)

This is the graph for the speedup calculated according to the number of threads used:



From the graph we can see that speedup increases as the number of threads increases but this growth is not linear. This happens because there is a load imbalance on the threads as some parts of the mandelbrot set need more computation than the others, which means some threads have to do more work than the others. We can also see a dip in the speedup when using 3 threads.

Part 3)

To confirm (or disprove) your hypothesis, measure the amount of time each thread requires to complete its work by inserting timing code at the beginning and end of `workerThreadStart()`. How do your measurements explain the speedup graph you previously created?

We measured the time taken for the thread to complete its work like this:

```
double start = CycleTimer::currentSeconds();
mandelbrotSerial(args->x0, args->y0, args->x1, args->y1, args->width, args->height, startRow,
endRow - startRow, args->maxIterations, args->output);
double end = CycleTimer::currentSeconds();
```

```
Hello from thread 4, finished in [0.021] sec
Hello from thread 0, finished in [0.026] sec
Hello from thread 3, finished in [0.108] sec
Hello from thread 1, finished in [0.111] sec
Hello from thread 2, finished in [0.170] sec
Hello from thread 0, finished in [0.019] sec
```

We can clearly see that some threads take less time than others. This confirms our hypothesis about imbalance in workload distribution among the threads.

Part 4)

Modify the mapping of work to threads to improve speedup to at about 7-8x on both views of the Mandelbrot set (if you're above 7x that's fine, don't sweat it). You may not use any synchronization between threads in your solution. We are expecting you to come up with a single work decomposition policy that will work well for all thread counts---hard coding a solution specific to each configuration is not allowed! (Hint: There is a very simple static assignment that will achieve this goal, and no communication/synchronization among threads is necessary.). In your writeup, describe your approach to parallelization and report the final 8-thread speedup obtained.

To increase the speedup, we used round robin technique while assigning rows to the threads. for (unsigned int row = threadIdx; row < args->height; row += numThreads) The loop ensures that every thread starts from the row corresponding to its thread ID. For example: If threadIdx = 0, it starts at row 0. Then each thread processes numThreads-th row. For example: if numThreads = 4, Thread 0 processes rows 0, 4, 8, 12...

This optimization balances the workload a little bit and increases the speedup. The max speedup we reached for view 1 was 6.46x

```
Wrote image file mandelbrot-thread.ppm
(6.46x speedup from 8 threads)
```

(resource used in this part:

<https://www.worldscientific.com/doi/10.1142/S0129626419500038?srsIid=AfmBOorn-0kWSblj8exT7LGKZ5OylnQ5DvpSalwpibE98gygnyA2BKF>)

Part 5)

Now run your improved code with 16 threads. Is performance noticably greater than when running with eight threads? Why or why not?

No, even with 16 threads the performance does not improve since the CPU only supports up to 8 threads (2 threads per core)

```
Wrote image file mandelbrot-thread.ppm
(5.83x speedup from 16 threads)
```

Question 2:

Part 1)

Implement a vectorized version of clampedExpSerial in clampedExpVector . Your implementation should work with any combination of input array size (N) and vector width (VECTOR_WIDTH).

Done on code

Part 2)

Run `./myexp -s 10000` and sweep the vector width from 2, 4, 8, to 16. Record the resulting vector utilization. You can do this by changing the `#define VECTOR_WIDTH` value in `CS149intrin.h`. Does the vector utilization increase, decrease or stay the same as `VECTOR_WIDTH` changes? Why?

Running `./myexp -s 1000` with different vector widths gave us the followings results:

Vector Width	Vector Utilization
2	83.1%
4	82.6%
8	82.1%
16	81.9%

```
***** Printing Vector Unit Statistics *****
Vector Width:      8
Total Vector Instructions: 2253
Vector Utilization: 82.1%
Utilized Vector Lanes: 14794
Total Vector Lanes: 18024
```

```
***** Printing Vector Unit Statistics *****
Vector Width:     16
Total Vector Instructions: 1137
Vector Utilization: 81.9%
Utilized Vector Lanes: 14904
Total Vector Lanes: 18192
```

This shows us that as the Vector Width increases, the utilization decreases. The cause of this inverse relation is that with a larger width value, the possibility of inactive lanes in the last iteration increases. Since the array of size N is processed in chunks of size Vector Width, if N is not divisible by the width then the last block might have inactive lanes. This means that these inactive lanes perform no useful task thus decreasing utilization.

Question 3:

Part 1)

Compile and run the program `mandelbrot ispc`. The ISPC compiler is currently configured to emit 8-wide AVX2 vector instructions. What is the maximum speedup you expect given what you know about these CPUs? Why might the number you observe be less than this ideal? (Hint: Consider the characteristics of the computation you are performing?)

Describe the parts of the image that present challenges for SIMD execution? Comparing the performance of rendering the different views of the Mandelbrot set may help confirm

your hypothesis.).

The ISPC compiler can emit 8-wide AVX2 vector instructions which means it can process 8 elements in parallel. So, in theory it gives up to 8x speedup, however the maximum speedup we got was **4.90x** for VIEW 1 and **4.13x** for VIEW 2. This happens due to the imbalanced computational load in the mandelbrot set image as some regions of the image need more work done as compared to others. For example the parts of the image where the white branches appear needs extra computation than the black or white parts. We know that SIMD execution of ISPC works best when the workload is divided equally among the vectors, which is not the case with this program. This brings the speedup down from the theoretical value

```
[mandelbrot serial]:          [186.755] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot ispc]:           [38.106] ms
Wrote image file mandelbrot-ispc.ppm
                             (4.90x speedup from ISPC)
```

Part 2)

1. Run `mandelbrot_ispc` with the parameter `--tasks`. What speedup do you observe on view 1? What is the speedup over the version of `mandelbrot_ispc` that does not partition that computation into tasks?

Speedup from task ISPC = 9.47x

Speedup from regular ISPC = 4.68x

```
[mandelbrot serial]:          [190.004] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot ispc]:           [40.599] ms
Wrote image file mandelbrot-ispc.ppm
[mandelbrot multicore ispc]:  [20.057] ms
Wrote image file mandelbrot-task-ispc.ppm
                             (4.68x speedup from ISPC)
                             (9.47x speedup from task ISPC)
```

2. There is a simple way to improve the performance of `mandelbrot_ispc --tasks` by changing the number of tasks the code creates. By only changing code in the function `mandelbrot_ispc_withtasks()`, you should be able to achieve performance that exceeds the sequential version of the code by over 32 times! How did you determine how many tasks to create? Why does the number you chose work best?

The best number of tasks is 8 since the machine we're working on has 4 cores (2 hyper threads per core). However, running the program with 8 tasks causes a 'segmentation dump' error. This is the change we made:

```

launch[8] mandelbrot_ispc_task(x0, y0, x1, y1,
                                width, height,
                                rowsPerTask,
                                maxIterations,
                                output);

```

It lead to this:

```

[mandelbrot serial]: [182.503] ms
Wrote image file mandelbrot-serial.ppm
[mandelbrot ispc]: [38.797] ms
Wrote image file mandelbrot-ispc.ppm
*** stack smashing detected ***: terminated
Segmentation fault (core dumped)

```

Question 4:

Part 1)

Build and run sqrt. Report the ISPC implementation speedup for single CPU core (no tasks) and when using all cores (with tasks). What is the speedup due to SIMD parallelization? What is the speedup due to multi-core parallelization?

```

[sqrt serial]: [593.633] ms
[sqrt ispc]: [121.185] ms
[sqrt task ispc]: [9.053] ms
                (4.89x speedup from ISPC)
                (65.57x speedup from task ISPC)

```

Part 2)

Modify the contents of the array values to improve the relative speedup of the ISPC implementations. Construct a specific input that maximizes speedup over the sequential version of the code and report the resulting speedup achieved (for both the with- and without-tasks ISPC implementations). Does your modification improve SIMD speedup? Does it improve multicore speedup (i.e., the benefit of moving from ISPC without-tasks to ISPC with tasks)? Please explain why.

We change inputs to range [0.5,1.5] which has the smallest and even iteration and it runs for 4 or less iterations for every value ,even iteration is friendly for SIMD.

```

[sqrt serial]: [284.398] ms
[sqrt ispc]: [22.649] ms
[sqrt task ispc]: [5.866] ms
                (12.556x speedup from ISPC)

```

(48.48x speedup from task ISPC)

SIMD executes multiple operations in parallel within a single instruction.

If the number of iterations (array size) is even, it aligns well with SIMD vector widths (e.g., 4, 8, or 16 elements per vector operation).

This avoids "tail processing," where some SIMD lanes are unused, leading to inefficiencies.

Avoids idling of processors as on each processor 4 or less iterations are run for every value. so almost equal processing on each processor. No processor sits idle.

Part 3)

Construct a specific input for sqrt that minimizes speedup for ISPC (without-tasks) over the sequential version of the code. Describe this input, describe why you chose it, and report the resulting relative performance of the ISPC implementations. What is the reason for the loss in efficiency? (keep in mind we are using the --target=avx2 option for ISPC, which generates 8-wide SIMD instructions).

ISPC vs. ISPC Tasks		
Feature	ISPC (without tasks)	ISPC (with tasks)
Parallelism	Uses SIMD (vectorization)	Uses SIMD + Multi-Core (parallelism across cores)
Execution Unit	Runs on a single CPU core but processes multiple elements per instruction (e.g., AVX2: 8 elements at a time)	Uses multiple CPU cores (each core runs SIMD instructions independently)
Data Processing	Works on a single segment of data at a time	Splits data into chunks across CPU cores
Synchronization	Implicit SIMD synchronization (all lanes execute together)	Task-based execution with thread scheduling
Best For	Compute-bound problems where vectorization helps	Large workloads that benefit from multi-core scaling

So we want to decrease ISPC without tasks. We can do something like giving it an input where half the instructions take too much time and the rest half get done instantly this will cause idling of lanes and ispc without tasks will run slow.

Ispc with tasks won't become slow as it will get different cores and work will be divided.

```
unsigned int counter = 0; // Tracks position in an 8-element block
```

```
for (unsigned int i = 0; i < N; i++) {  
    if (counter >= 5 || counter == 0) {  
        values[i] = 2.999f;  
    } else {  
        values[i] = 1.0f;  
    }  
}
```

```

    }

    counter++; // Move to the next position in the block

    if (counter == 8) {
        counter = 0; // Reset every 8 iterations
    }
}

```

Question 5

PART 1)

Compile and run saxpy. The program will report the performance of ISPC (without tasks) and ISPC (with tasks) implementations of saxpy. What speedup from using ISPC with tasks do you observe? Explain the performance of this program. Do you think it can be substantially improved? (For example, could you rewrite the code to achieve near linear speedup? Yes or No? Please justify your answer.)

```

[saxpy serial]: [8.600] ms [47.533] GB/s [7.458] GFLOPS
[saxpy ispc]: [6.936] ms [53.521] GB/s [7.947] GFLOPS
[saxpy task ispc]: [6.349] ms [58.643] GB/s [8.555] GFLOPS
                (1.07x speedup from use of tasks)

```

Each element of the output is computed independently. We could divide arrays and give to different threads to do the work. Like Q1. but memory bandwidth is the bottleneck, therefore tasks have no improvement. There is memory access 3 times in each line of execution so it cant be improved more than that.

Modern CPUs have high floating-point performance but memory access is slow so whatever we do we cant achieve more speedup than that because of this problem. We would need to decrease memory access in each line.

Even with multiple CPU cores, all cores compete for the same memory bandwidth, leading to the same returns from parallelization. since all cores share the same memory bandwidth, increasing cores does not proportionally increase speed. So parallelization will have no effect.